

Progress Report – Real-Time Programming

Lab time: friday 8-12, workstation 11, group 82

Jens Tjomsland

Mathias Sagerup

Gustav Smidtsrod

jenstj@ntnu.no

mathisag@ntnu.no

gustavsm@ntnu.no

February 1, 2026

For å unngå ett enkelt feilpunkt og å støtte partial failure har vi valgt en peer-to-peer-arkitektur fremfor master-slave. Om alle nodene har samme situasjonsforståelse og samme logikk vil de være enige om hvem som skal gjøre hva, uten at en master må tilordne ordre over nettverket. Dette gir et OR-system der hall calls kan overtas dersom en node blir utilgjengelig. Hver heis er delt inn i moduler som vist i figur 1. Communication håndterer nettverksmeldinger, FaultHandler avgjør hvilke heiser som er tilgjengelige og håndterer feiltilstander, HallCallAssigner fordeler hall calls lokalt til heisen, og ElevatorState eier heislogikken og styrer maskinvaren via Driver (I/O). Denne inndelingen gir høy kohesjon og lav kobling, og gjør systemet robust mot partial failure.

En utfordring er å få heiser til å bli enige om systemtilstand over et upålitelig nettverk. Vi har derfor valgt at hver heis gjør periodisk kringkasting av en standardisert melding med UDP. Denne meldingen inneholder avsender-ID, lokal heistilstand, sitt syn på hall calls, sitt syn på andre heisers tilstander og om heisen er i stand til å gjennomføre ordre. Meldinger i feil rekkefølge og duplikate meldinger håndteres ved å anta at den siste mottatte meldingen er den riktige. Periodisk sending gjør at oppdatert informasjon vil nå frem over kort tid selv ved pakketap. Cab calls inngår i heistilstanden slik at om en ufortsett restart skjer, så vil heisen kunne finne tilbake til disse. En heis oppdaterer kun sitt bilde av en annen heis sin tilstand basert på meldinger fra denne heisen. Denne regelen løser konflikter der heisene er uenige i hvilken tilstand en gitt heis befinner seg i. Hall calls modelleres som tilstandsoverganger (false $\rightarrow$ true $\rightarrow$ completed $\rightarrow$ false) som vist i figur 2, noe som håndterer negasjon og sikrer enighet om hvilke ordre som finnes. Siden vi antar at enhver heis enten er tilkoblet nettet og derfor er i stand til å kommunisere nye hall calls, eller er i stand til å utføre lokalt oppfattede hall calls, ser vi ingen behov for ordrebekreftelse. Enten kommuniseres ordenen riktig, ellers lagres og utføres den i værste fall lokalt. Dette systemet sikrer light contract for hall calls.

En sentral utfordring er å avgjøre hvilke heiser som til enhver tid er i stand til å utføre hall calls, slik at bestillinger kan delegeres riktig selv ved partial failure. I vårt design regnes en heis som utilgjengelig enten dersom nettverkstilkobling forsvinner, eller dersom heisen oppdager at den ikke klarer å gjennomføre sine tilegnede ordre. Hver node har en lokal FaultHandler som kjenner til heisenes tilgjengelighet. Denne vet hvilke heiser som er tilkoblet nettet ved å sjekke timere som nullstilles hver gang en melding mottas fra hver heis. En timer brukes også for å sjekke hvor lenge siden den lokale heisen har gjennomført en ordre, gitt at aktive ordre finnes. Om en heis innser at den ikke er i stand til å gjennomføre ordre ved for eksempel motorfeil vil dette kommuniseres til de andre heisene. Resultatet er et sett av tilgjengelige heiser som brukes direkte av HallCallAssigner til å fordele hall calls kun mellom noder som faktisk kan utføre dem. Dersom en heis blir alene i nettverket, går den i lokal modus der den glemmer gamle hall calls, og stoler på at de andre nodene gjennomfører disse. Den fortsetter å gjennomføre nye lokalt tilegnede hall calls og cab calls. Dette systemet sikrer en global enighet om hvilke heiser hall calls skal fordeles mellom.

I et distribuert heissystem må noder kunne forsvinne og komme tilbake uten at kall går tapt. Når en heis kommer tilbake på nett, skiller vi mellom to tilfeller: enten har den mistet kjennskap til lokal tilstand etter restart, eller så har den fortsatt gyldig lokal tilstand etter et nettverksbrudd. I siste tilfellet legger FaultHandler merke til at heisen mister eller gjenoppretter nettverkstilkobling, og håndterer riktig deling av utdatert data. Ved en eventuell restart går heisen inn i en reinitialiseringsfase der lokal systemtilstand overskrives basert på informasjon mottatt fra andre heiser. Slik gjenopprettes cab calls uten at brukeren må trykke på nytt. Dersom heisen har fungert lokalt mens den var frakoblet, behandles dens aktive hall calls som gyldige ved gjeninnkobling og aktive hall calls deles med de andre heisene. Systemet sikrer at gjenværende noder tar over ved partial failure og riktig deling av data ved en restart eller tilbakekobling på nettet.

Ved normal drift følger programmet en flyt der informasjon først oppdateres basert på input og meldinger fra andre heiser, ordre tilordnes, og heisen setter riktig output for å gjennomføre tilegnede ordre. Til slutt overføres lokal informasjon til kommunikasjonsmodulen for sending. Dette skjer i en hovedløkke i en samlet go-routine. En annen go-routine sender og mottar meldinger på nettverket og legger nødvendig informasjon systemetisk i buffere. Den siste go-routinen gjør feilhåndtering. Dette gjøres i en egen go-routine slik at den kan ordne systemet selv om hovedløkken feiler. Deler av programflyten er beskrevet i flytdiagrammer. Disse er ikke helt komplette, men gir en indikasjon på hvordan programmet skal fungere.

1 Figurer

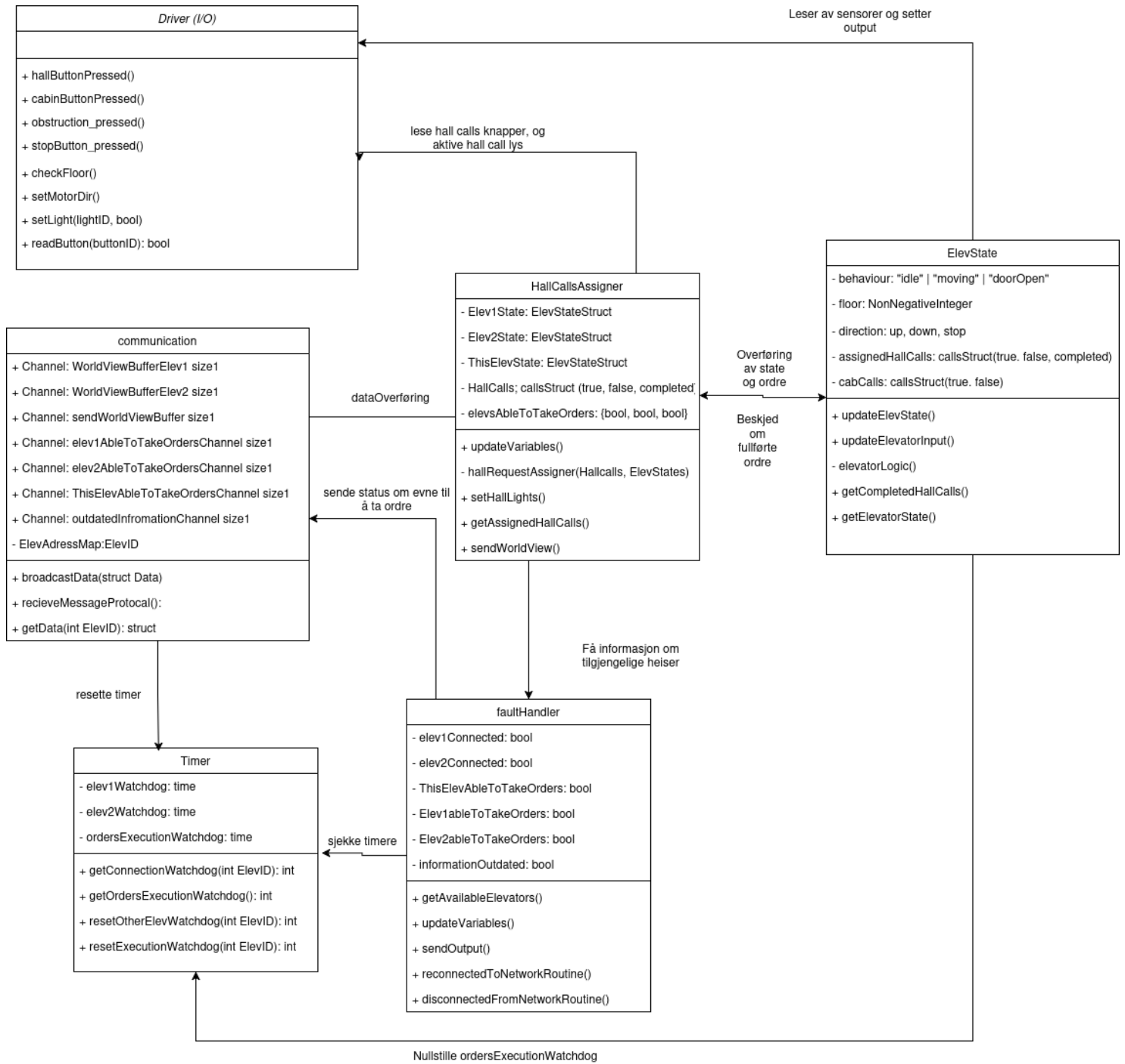


Figure 1: klassesdiagram

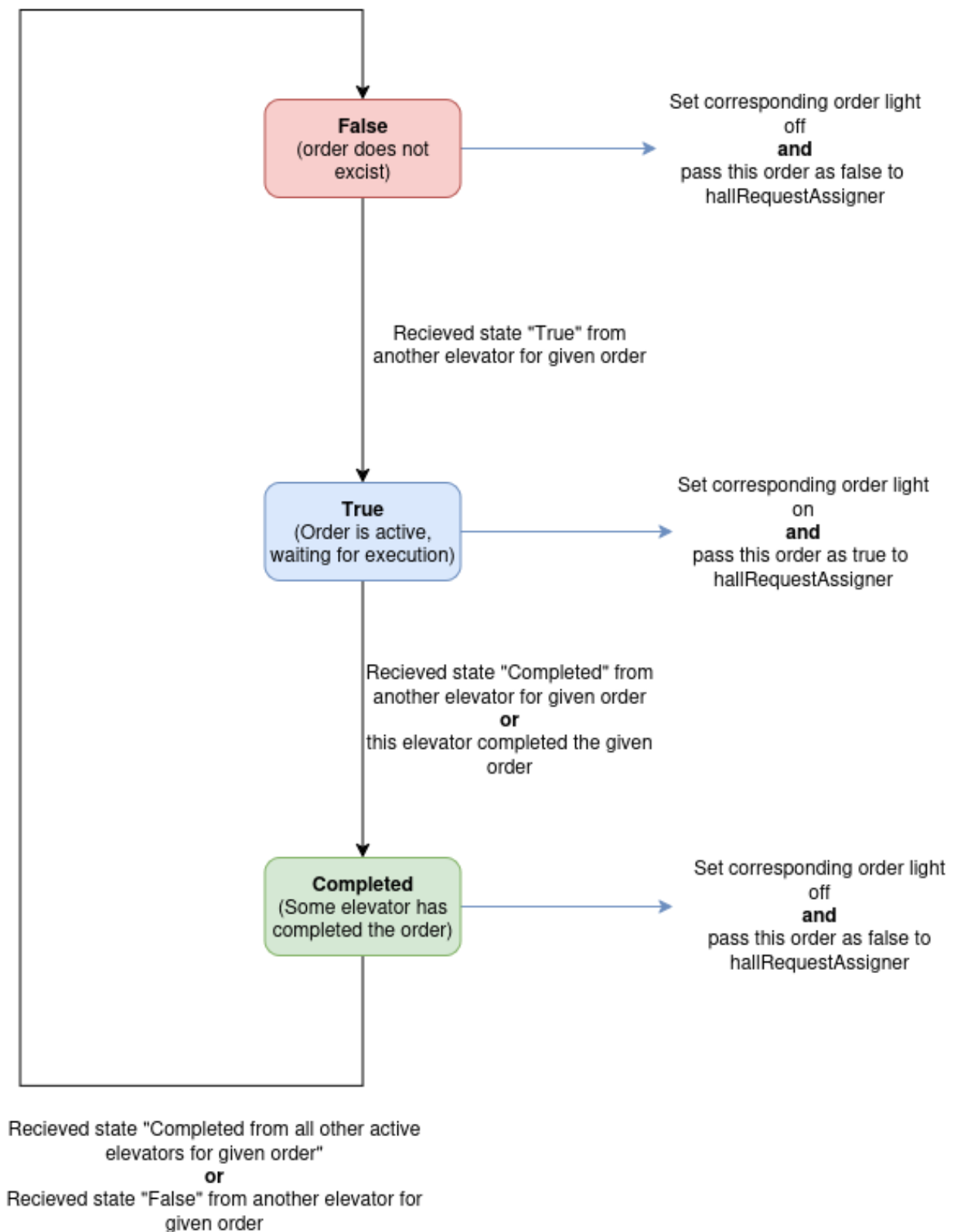


Figure 2: Tilstander hallcalls

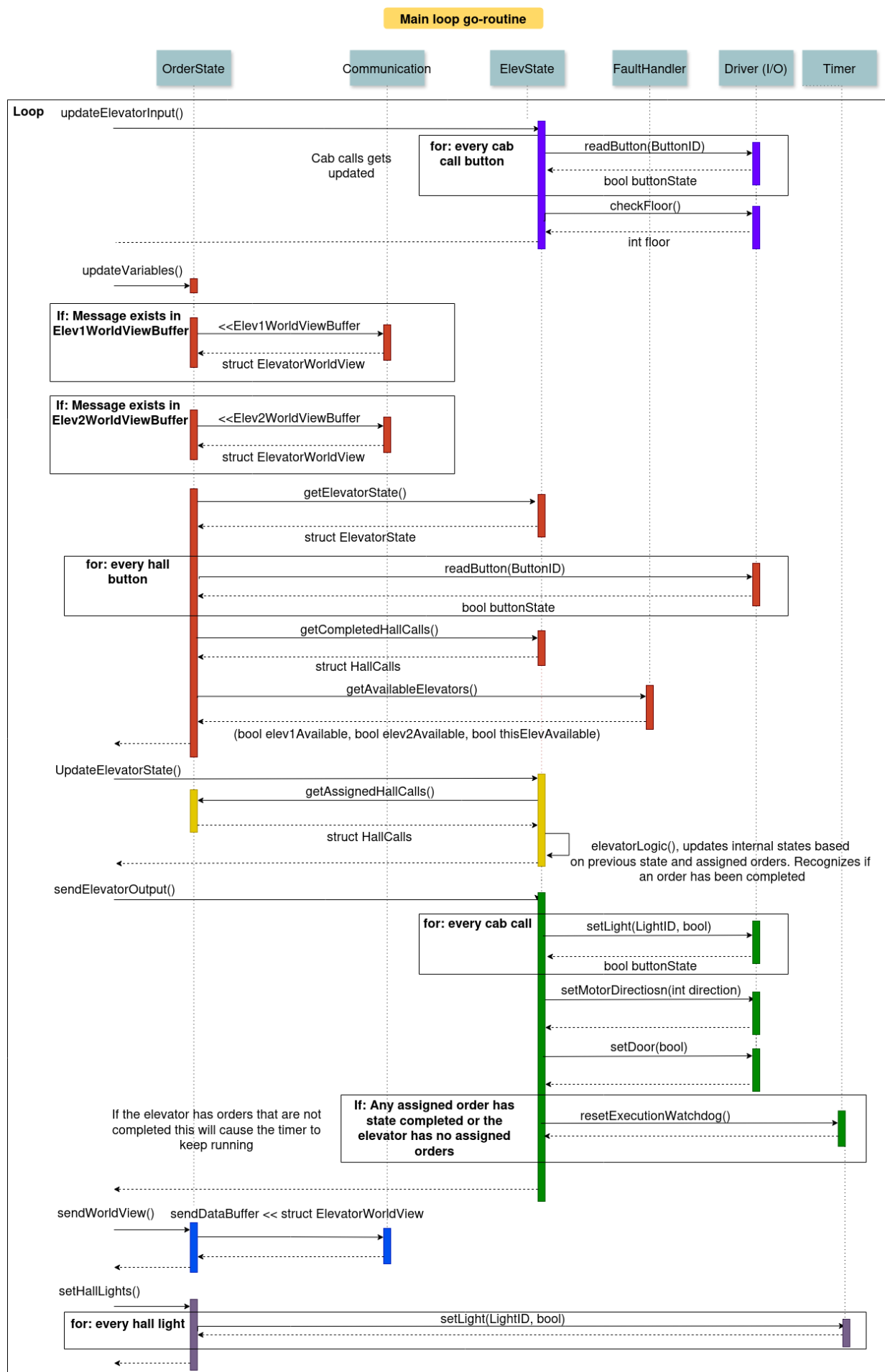


Figure 3: Sekvensdiagram for hovedloopen i programmet

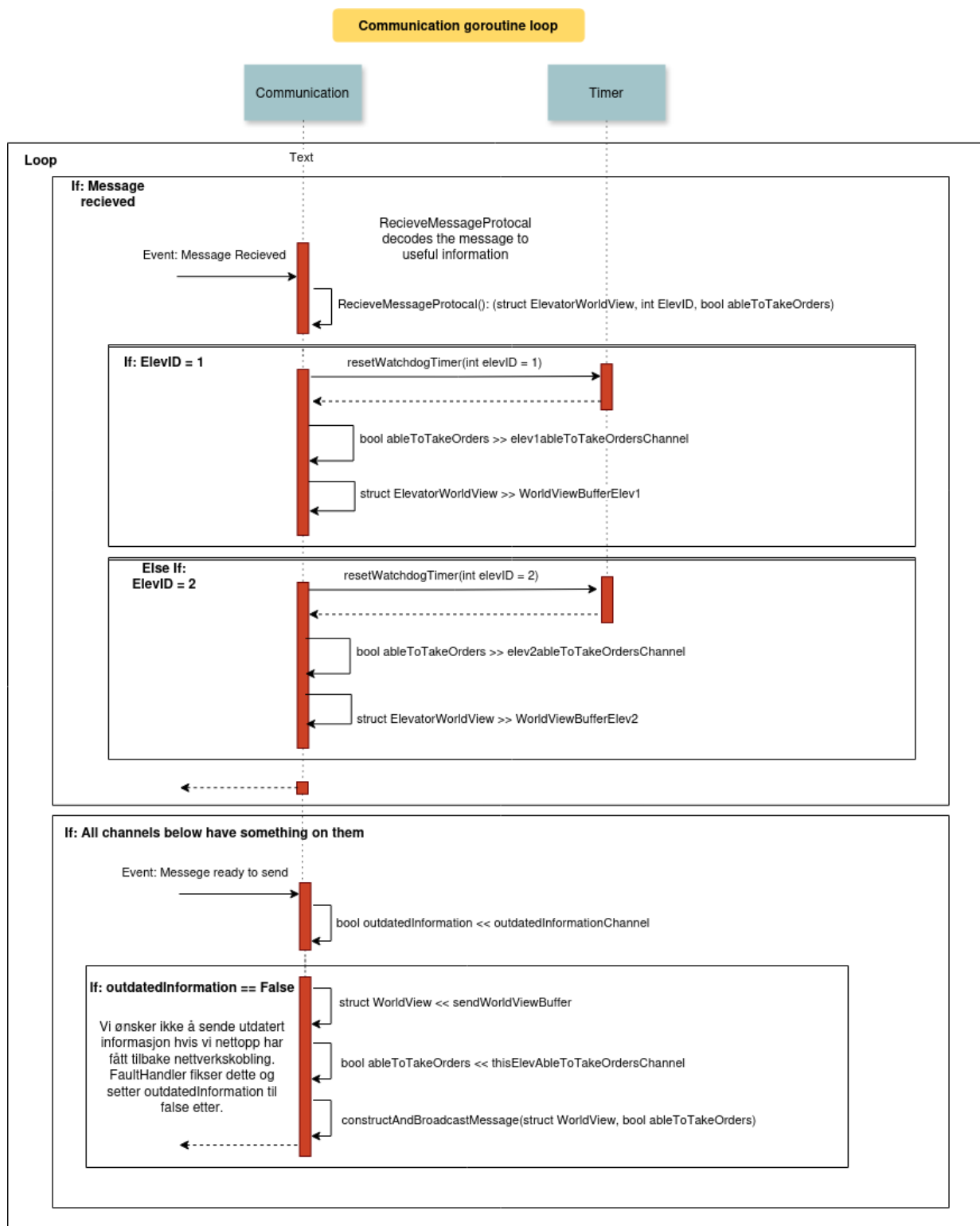


Figure 4: Sekvensdiagram for kommunikasjon-loopen

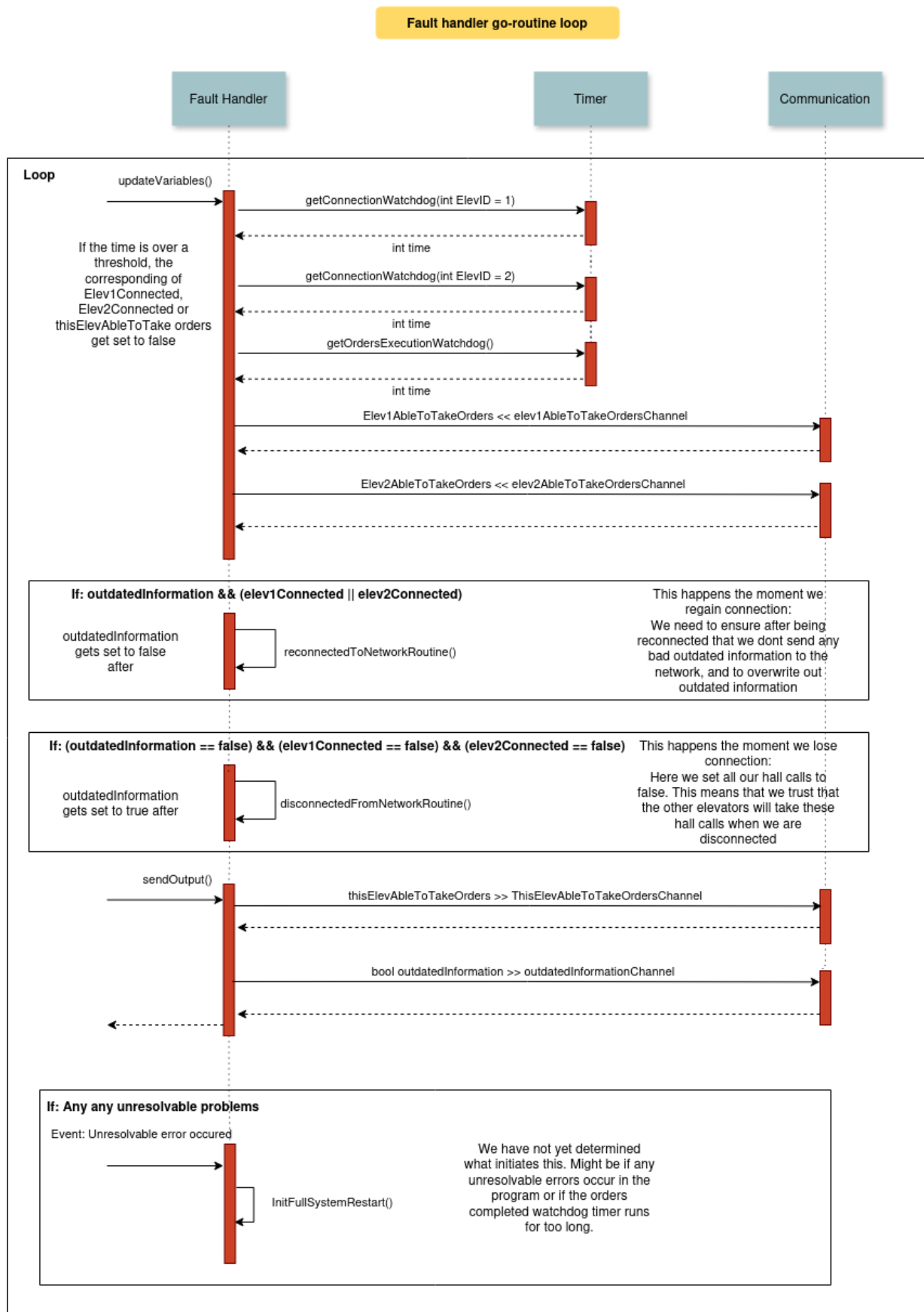


Figure 5: Sekvensdiagram for fault handler loop